



Assignment 2

Computernetze und ihre Leistung

von:

Michael Heise

Linus Henn

Luca Lars Stoever

Matrikelnr.: 537385

Matrikelnr.: 576038

Matrikelnr.: XXXXX

Kurs:

Computernetze und ihre Leistung SoSe 2026

Dozent: **Ralph-Günther Holz**

Münster, 20. Juli 2026

Contents / Inhaltsverzeichnis

1	Aufbau	1
2	Methodik	3
3	UDP	3
3.1	Durchführung	3
3.2	Ergebnisse	4
3.3	Diskussion	5
4	TCP	6
4.1	Parameter und Durchführung	6
4.2	Ergänzende Methodik	6
4.3	Ergebnisse	6
4.4	Diskussion	6
5	TCP unter Jitter	6
5.1	Parameter und Durchführung	6
5.2	Ergänzende Methodik	6
5.3	Ergebnisse	6
5.4	Diskussion	6
6	Vergleich von TCP Reno und TCP Cubic	6
6.1	Parameter und Durchführung	7
6.1.1	Gemeinsames Experiment-Setup	7
6.1.2	Durchführung mit TCP Reno	7
6.1.3	Durchführung mit TCP Cubic	7
6.1.4	Messdaten	7
6.2	Ergänzende Methodik	7
6.2.1	Durchsatzanalyse	7
6.2.2	Analyse des Staukontrollfensters	7
6.2.3	Retransmissionen und Paketverluste	7
6.2.4	Auswertungsskripte	7
6.3	Ergebnisse	7
6.3.1	Durchsatz	7
6.3.2	Entwicklung des Staukontrollfensters	7
6.3.3	Retransmissionen, Paketverluste und deren Einfluss	7
6.3.4	Zusammenfassender Vergleich	7
6.4	Diskussion	7

6.4.1	Wachstum und Reaktion	7
6.4.2	Performance, Robustheit und Stabilität	7
6.4.3	Praktische Einsatzszenarien	7
6.4.4	Grenzen des Experiments	7

1 Aufbau

Als Grundlage der hier durchgeführten Experimente dient ein virtuelles Netzwerk ausgeführt in Docker. Dabei werden zwei Hosts (n1, n2) miteinander verknüpft. Es ist möglich, TCP und UDP Verbindungen aufzubauen und die übertragenen Pakete in eine PCAP Datei zu speichern. Zudem können einige Parameter für den Link zwischen den Hosts eingestellt werden.

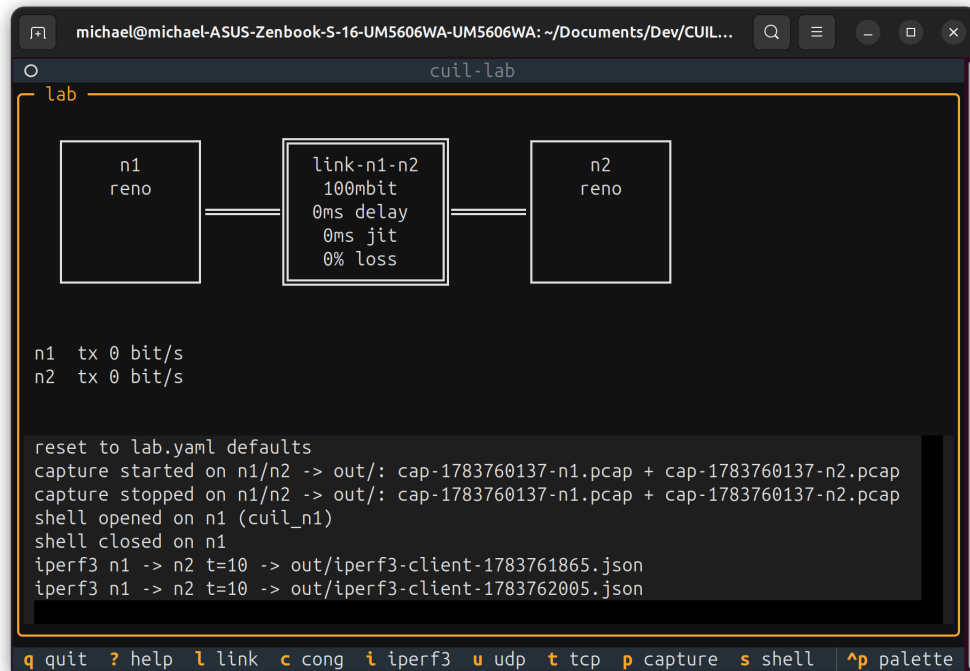


Abbildung 1: Docker Netzwerk

Link Einstellungen

Der Link zwischen den Hosts kann in der Bandbreite, Latenz, Verzögerung und Paketverlust konfiguriert werden.

Standardmäßig ist der Link mit folgenden Parametern konfiguriert:

- Bandwidth: 100 Mbit/s
- Delay: 0 ms
- Jitter: 0 ms
- Paketverlust: 0 %
- Burst: 32kbit

TCP Staukontrolle

Für jeden Host kann ein Standard für die Staukontrolle gewählt werden. Zur Auswahl stehen TCP Reno und TCP CUBIC. Uns ist nicht bekannt, welcher Standard genutzt wird, da diese kontinuierlich weiterentwickelt werden. Folgende Quellen geben einen Überblick über die beiden Staukontrollen, auch wenn sich die genaue Version des Standards unterscheiden kann.

Standardmäßig ist die Staukontrolle auf TCP Reno eingestellt.

- TCP Reno: [2]

- TCP CUBIC: [4]

iperf3

iperf3 ist ein Tool, um Netzwerkleistung in IP Netzwerken zu messen. Das Programm misst den Durchsatz, Paketverlust und andere Parameter [1]. In der Docker Umgebung muss ein Host als Server und ein Host als Client angegeben werden anschließend noch eine Messdauer und ein Messintervall. Eine Messung erzeugt eine JSON und eine Log-Datei, die die Ergebnisse der Messung enthalten. In der Log-Datei ist zu sehen, dass für jedes Intervall gemessen wird wie viele Daten übertragen wurden, daraus ergibt sich der gemessene Durchsatz und die Übertragungsrate für den Link.

udp

Das UDP Tool in der Docker Umgebung kann genutzt werden, um UDP Verbindungen zwischen den Hosts aufzubauen. Dabei müssen Server und Client angegeben werden, sowie die Anzahl der Pakete, das Intervall und die Paketgröße. Beim Ausführen sendet der Client UDP Pakete an den Server. Für den ganzen Vorgang wird eine Log-Datei pro Endpunkt erstellt, die die Anzahl der gesendeten und empfangenen Pakete enthält.

Die Standardeinstellungen für das UDP Tool sind:

- Server: n2
- Client: n1
- Count: 10
- Interval: 0.1 s
- size: 64 B

tcp

Das TCP Tool in der Docker Umgebung kann genutzt werden, um TCP Verbindungen zwischen den Hosts aufzubauen. Dabei müssen Server und Client angegeben werden, sowie die Anzahl der Pakete, das Intervall und die Paketgröße. Zusätzlich zu diesen Einstellungen, die wir auch für UDP nutzen, kann eingestellt werden, ob eine persistente Verbindung genutzt werden soll, oder ob für jedes Paket eine neue Verbindung aufgebaut werden soll.

Die Standardeinstellungen für das TCP Tool sind:

- Server: n2
- Client: n1
- Count: 10
- Interval: 0.1 s
- size: 64 B
- Mode: persistent

PCAP Capture

In der Docker Umgebung ist es möglich, die übertragenen Pakete in eine PCAP Datei zu speichern. Dazu kann ein Capture mit p gestartet werden. Anschließend wird pro Host eine PCAP Datei erstellt, die alle Pakete enthält, die über den Link übertragen wurden bis das Capture wieder gestoppt wird.

Shell

In der Docker Umgebung ist es möglich, eine Shell auf den Hosts zu öffnen. Mit der Shell können Hosts konfiguriert und Befehle ausgeführt werden, die nicht über die Docker Umgebung möglich sind.

2 Methodik

Für die Experimente wurde das im Aufbau 1 beschriebene virtuelle Netzwerk genutzt. Dabei wurde für jedes Experiment eine Datei erzeugt, die den Datenverkehr im Netzwerk aufzeichnet. iperf3 1 erzeugt JASON und TXT-Dateien, die direkt die Ergebnisse der Messung enthalten. Teilweise werden auch Shell Befehle 1 genutzt, um die Experimente zu starten und zu stoppen oder um Ausgaben der Experimente zu erhalten. PCAP Dateien werden von dem 1 beschriebenen Capture Tool erzeugt und können anschließend mit Wireshark [3] analysiert werden hinsichtlich bestimmter Fragestellungen untersucht werden. Wireshark ist ein Tool, um Netzwerkverkehr zu analysieren und kann PCAP Dateien öffnen und die enthaltenen Pakete darstellen. Einzelne Pakete können byteweise analysiert werden. Wireshark erlaubt einzelne Parameter und Flags der Verbindungen zu analysieren und die Pakete nach bestimmten Kriterien zu filtern.

3 UDP

Das UDP (User Datagram Protocol) ist ein verbindungsloses Protokoll der Transportschicht. In diesem Versuch soll der verlustfreie und verlustbehaftete UDP Datenverkehr zwischen zwei Hosts untersucht werden.

3.1 Durchführung

Für die Durchführung wurde das Docker Netzwerk 1 benutzt. Es wurde zweimal der UDP Datenverkehr zwischen zwei Hosts untersucht. Die Docker Umgebung wurde bei der Standardkonfiguration belassen. Einzig die Link-Parameter 1 wurden beim zweiten Durchlauf angepasst, um den Paketverlust zu simulieren:

- Bandwidth: 100 Mbit/s
- Delay: 0 ms
- Jitter: 0 ms
- Paketverlust: 30 %
- Burst: 32kbit

In beiden Fällen wurden folgende Schritte durchgeführt:

1. Starten der Docker Umgebung
2. Einstellen der Link Eigenschaften 1
3. Starten des Netzwerkverkehr-Mitschnitts 1
4. Starten der UDP Verbindung zwischen den Hosts n1 (UDP Client) und n2 (UDP Server) 1
 - Server: n2
 - Client: n1
 - Count: 30
 - Interval: 1 s
 - size: 64 B

5. Beenden des Netzwerkverkehr-Mitschnitts 1
6. Schließen der Docker Umgebung

Nachdem die Log- und PCAP-Dateien erstellt wurden, wurden diese ausgewertet und die Ergebnisse dokumentiert.

3.2 Ergebnisse

Die Log- und PCAP-Dateien treffen dieselben Aussagen über den UDP Datenverkehr zwischen den Hosts. Da die Log-Dateien weniger unnötige Informationen enthalten, werden diese für die Auswertung genutzt. Die PCAP-Dateien enthalten neben den UDP Paketen auch ARP Anfragen und Antworten, die für die Auswertung nicht relevant sind. Bei dem Link ohne Paketverlust wurden alle 30 Pakete erfolgreich mit nahezu keiner Verzögerung übertragen. Bei dem Link mit Paketverlust kamen nur 22 der 30 Pakete beim Server an. Die verlorenen Pakete entsprechen wie erwartet dem eingestellten Paketverlust von 30 %. Die verlorenen Pakete wurden erkannt, da sie vom Client gesendet wurden, aber nicht in der Log-Datei des Servers auftauchten.

Paket Nummer	Gesendet durch Client	Empfangen durch Server
1	J	J
2	J	J
3	J	N
4	J	J
5	J	J
6	J	J
7	J	J
8	J	J
9	J	N
10	J	J
11	J	J
12	J	N
13	J	N
14	J	N
15	J	N
16	J	J
17	J	J
18	J	J
19	J	N
20	J	J
21	J	N
22	J	J
23	J	J
24	J	J
25	J	J
26	J	J
27	J	J
28	J	J
29	J	J
30	J	J

Tabelle 1: Übertragungsstatus der Pakete zwischen Server und Client mit 30 % Paketverlust

Den Logs lässt sich außerdem entnehmen, dass in jeden Fall genau 30 Pakete vom Client gesendet wurden. Die verlorenen Pakete wurden nicht erneut gesendet, da UDP ein verbindungsloses Protokoll ist und keine Mechanismen zur Fehlerbehandlung implementiert sind.

3.3 Diskussion

Als Protokoll der Transportschicht wird der UDP Payload an die Anwendungsschicht weitergeleitet. Geht ein Paket verloren, so wird die Anwendungsschicht nicht darüber informiert. Die Anwendungsschicht kann also durch verlustbehafteten UDP Datenverkehr beeinflusst werden und muss gegebenenfalls selbst Mechanismen implementieren, um mit Paketverlusten umzugehen.

Auswirkung auf Anwendungsschicht

Die Anwendungsschicht erhält durch das UDP Protokoll lückenhafte Daten. Auswirkungen können unkritisch sein. Bei Video Streaming oder Voice over IP (VoIP) würde die Stimme oder das Bild kurzzeitig ausfallen oder verzerrt werden, aber die Anwendung würde weiterlaufen. Online Spiele, die auf UDP basieren, würden ebenfalls kurzzeitig einfrieren oder verzerrt dargestellt werden, aber die Anwendung würde weiterlaufen. Außerdem wäre ein erneutes Senden nutzlos, da die verlorenen Pakete nicht mehr relevant sind. Webseiten oder Dateiübertragungen, die über UDP mit Verlust geladen werden, wären unvollständig, und funktional eingeschränkt. Die Anwendung würde nicht mehr korrekt funktionieren, da die verlorenen Pakete erneut gesendet werden müssten, um die Funktionalität der Anwendung zu gewährleisten. Die Anwendungsschichtprotokolle können unterschiedlich auf Paketverlust reagieren.

Umgang mit Paketverlust

Bei Anwendungen bei denen alte Pakete nicht mehr verwendet werden können, oder ein Qualitätsverlust in Kauf werden kann, wird der Paketverlust toleriert.

Bei Anwendungen bei denen die verlorenen Pakete erneut gesendet werden müssen, um die Funktionalität der Anwendung zu gewährleisten, muss der Paketverlust erkannt und die verlorenen Pakete erneut gesendet werden. Einige Protokolle wie Quick UDP Internet Connections (QUIC) implementieren Flusskontrolle, Staukontrolle, Datenintegrität, aber ebenfalls Retransmission von verlorenen Paketen auf der Anwendungsschicht. Es werden ähnliche Mechanismen wie bei TCP implementiert, um die verlorenen Pakete erneut zu senden. Eine Kombination von Sequenznummern und Bestätigungen, sowie Timeouts, werden genutzt, um verlorene Pakete zu erkennen und erneut zu senden.

4 TCP

4.1 Parameter und Durchführung

4.2 Ergänzende Methodik

4.3 Ergebnisse

4.4 Diskussion

5 TCP unter Jitter

5.1 Parameter und Durchführung

5.2 Ergänzende Methodik

5.3 Ergebnisse

5.4 Diskussion

6 Vergleich von TCP Reno und TCP Cubic

Hinweis zur Gliederung: Die Darstellung folgt dem Ablauf des Experiments und weicht deshalb von der Reihenfolge der Teilaufgaben a bis d ab. Die Teilaufgaben 3a und 3b werden in Abschnitt 6.1 behandelt. Die Analyse aus Teilaufgabe 3c wird in Abschnitt 6.2 beschrieben. Vergleich und Bewertung gemäß Teilaufgabe 3d erfolgen in den Abschnitten 6.3 und 6.4.

- 6.1 Parameter und Durchführung
 - 6.1.1 Gemeinsames Experiment-Setup
 - 6.1.2 Durchführung mit TCP Reno
 - 6.1.3 Durchführung mit TCP Cubic
 - 6.1.4 Messdaten
- 6.2 Ergänzende Methodik
 - 6.2.1 Durchsatzanalyse
 - 6.2.2 Analyse des Staukontrollfensters
 - 6.2.3 Retransmissionen und Paketverluste
 - 6.2.4 Auswertungsskripte
- 6.3 Ergebnisse
 - 6.3.1 Durchsatz
 - 6.3.2 Entwicklung des Staukontrollfensters
 - 6.3.3 Retransmissionen, Paketverluste und deren Einfluss
 - 6.3.4 Zusammenfassender Vergleich
- 6.4 Diskussion
 - 6.4.1 Wachstum und Reaktion
 - 6.4.2 Performance, Robustheit und Stabilität
 - 6.4.3 Praktische Einsatzszenarien
 - 6.4.4 Grenzen des Experiments

Literatur

- [1] Energy Sciences Network (ESnet). iperf3 documentation, 2026. Accessed: 2026-07-11. URL: <https://software.es.net/iperf/>.
- [2] Andrei Gurtov, Tom Henderson, Sally Floyd, and Yoshifumi Nishida. The NewReno Modification to TCP's Fast Recovery Algorithm. RFC 6582, April 2012. URL: <https://www.rfc-editor.org/info/rfc6582>, doi:10.17487/RFC6582.
- [3] Wireshark Foundation. Wireshark, 2026. Accessed: 2026-07-11. URL: <https://www.wireshark.org/>.
- [4] Lisong Xu, Sangtae Ha, Injong Rhee, Vidhi Goel, and Lars Eggert. CUBIC for Fast and Long-Distance Networks. RFC 9438, August 2023. URL: <https://www.rfc-editor.org/info/rfc9438>, doi:10.17487/RFC9438.